2. Gyakorlat

Algoritmusok és adatszerkezetek 2.

Lempel-Ziv-Welch/LZW módszer

Alapok

- Az LZW kódolás egy kezdeti kódtáblát bővít
- A szöveg bizonyos szavaiból szótárat épít
  - Egyre hosszabb már „látott” szavakhoz rendel új kódszót
- Ez szavak egy halmaza, amit S-sel jelölünk
- A szótárról három dolgot tételezünk fel:
  - az egybetűs szavak mind szerepelnek benne;
  - ha egy szó benne van a szótárban, akkor annak minden kezdődarabja is benne van;
  - a tárolt szavaknak fix hosszúságú kódjuk van; $x \in S$ szó kódját c(x) jelöli.

LZW Algoritmus

1. Olvassuk a bemenő állomány következő betűjét K-ba.
2. Ha az előző olvasási kísérlet sikertelen volt (vége a bemenetnek), akkor írjuk ki c(z)-t, és álljunk meg.
3. Ha a zK szó is S-ben van, akkor z ← zK, és menjünk vissza (1)-re.
4. Különben (ha zK ∉ S) írjuk ki c(z)-t, tegyük a zK szót S-be. Legyen z ← K, majd menjünk vissza (1)-re.

1. Feladat

Legyen a tömörítendő szöveg: ABAB ABAA CAAA AAAA A

Kezdeti kódtábla

Karakter	kód
A	1
B	2
C	3

Megoldás

- A kódolás során a bemenetet pontosan egyszer olvassuk végig úgy, hogy mindig a már ismert (kóddal rendelkező) leghosszabb szót keressük. Ha megtaláltuk, akkor
  - kiírjuk a talált szó kódját a kimenetre, és
  - bővítjük a kódszavak halmazát az αK szó képével, ahol az α a talált szó és K a következő karakter.
- Kezdetben a leghosszabb „ismert” szó az A, ennek kódja 1 és az új kóddal rendelkező szó az AB a 4 kóddal.
- A kódolás folyamán ezt az lépést ismételjük, amíg a szöveg végére nem érünk.

Kódoláshoz egy táblázat

Lépés	z	K	z.K	Ismert?	Todo

Ha nem ismert:

```
z.K → Kódtábla; Kiír(Kódtábla[z]); z:=K.
```

Ha ismert:

```
z:=z.K;
```

kód	aktuális szó	következő karakter	új kód
1	A	B	4
2	B	A	5
4	AB	A	6
6	ABA	A	7
1	A	C	8
3	C	A	9
1	A	A	10

Folyt.

8	AC	C	11
3	C	B	12
2	B	B	13
5	BA	A	14
10	AA	A	15
15	AAA	A	16
15	AAA	-	-

Dekódolás

Stratégia

- Mit kell dekódolni:
  - Nem kapjuk meg az egész kódtáblát, hanem csak a kódolt szöveget és a karakterek kódjait.
- Hogyan csináljuk akkor?
  - Útközben építjük a kódtáblát ugyanúgy, mint a kódoláskor.

Javasolt algoritmus

1. Inicializáljuk a kódtáblánkat a karakterek kódjaival; Y := NULL.
2. Olvassuk be a kód következő kódját, legyen ez X.
3. Ha az X üres, akkor VÉGE
4. S := Kód[X]; // nézzük meg minek a kódja az X
5. Kiír(S); // írjuk ki
6. Ha (Y != NULL) Kód[Y].Kód[X][0] → Kód[]; // Y az előzőleg olvasott kód
7. Y := X;
8. Ugorjunk a 2-es pontra

2. Feladat

Dekódoljuk az alábbi kódsorozatot, ha a kiinduló kódtáblánk: A=1, B=2

Kód: 1,2,3,5

Megoldás

1,2,3 ,5???

A,B,AB,

Kódtábla:

A=1,B=2,AB=3,BA=4,???=5

Nem tudjuk megoldani… Vagy mégis?

Folytatás

Hogyan jött létre az 5-ös kódszó?

A 3-as kód után olvastunk egy karaktert és az bekerült a kódtáblába, majd rögtön az következett az AB után a kódoláskor.

5-ös kód: ABx formájú. Mi lehet az x? Csakis A lehet ha rögtön a bekerülés után ki is írtuk.

3. Feladat

A tapasztalatok alapján javítsuk az algoritmust, hogy ezeket az eseteket is képes legyen kezelni.

Megoldás

1. Inicializáljuk a kódtáblánkat a karakterek kódjaival; Y := NULL.
2. Olvassuk be a kód következő kódját, legyen ez X.
3. Ha X üres, akkor vége!
4. Benne van ez a kódtáblában X?
    a. Ha igen
        i. S := Kód[X]; Kiír(S);
        ii. Ha (Y != NULL) Kód[Y].Kód[X][0] → Kód[];
    b. Ha nem
        i. S:=Kód[Y].Kód[Y][0];
        ii. Kiír(S)
        iii. S → Kód[];
    c. Y := X;

4. Feladat

Folytassuk a 2. Feladatban adott feladatot és dekódoljuk az alábbi kódot:

Kód: 1,2,3,5,6

A kiinduló táblánk: A=1, B=2

Játszadozni, művelődni

https://www.dcode.fr/lzw-compression

https://www.geeksforgeeks.org/lzw-lempel-ziv-welch-compression-technique/

https://www.youtube.com/watch?v=j2HSd3HCpDs

AVL fák

Ismétlés - Bináris keresőfa

Egy bináris fát keresőfának nevezünk, ha minden belső csúcsára és annak y kulcsára igazak az alábbi követelmények:

- A csúcs bal részfájában tetszőleges csúcs x kulcsára $x < y$.

- A csúcs jobb részfájában tetszőleges csúcs z kulcsára $z > y$

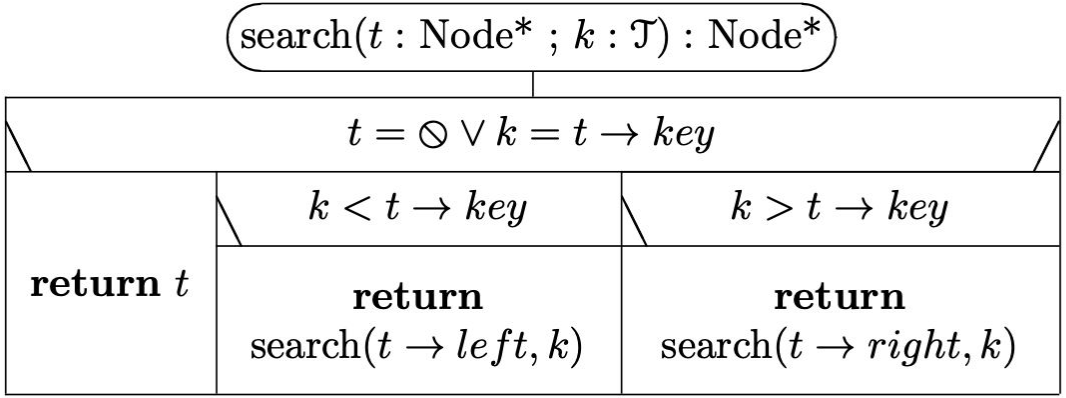
Note: rendezőfa: ugyanez a definíció, de megengedjük a $\leq$ és $\geq$, azaz keresőfában egyediek a kulcsok, rendezőfában lehetnek duplikáltak.

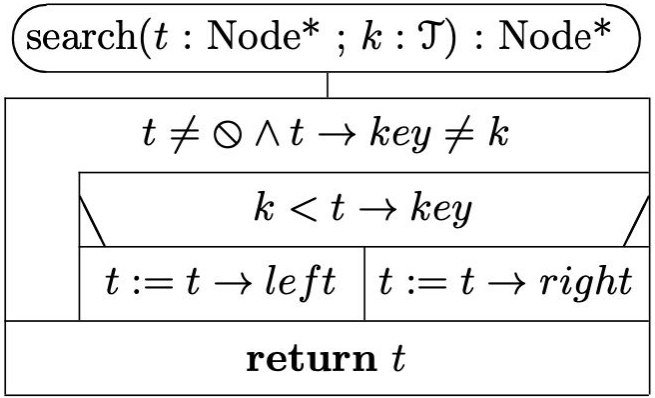
Ismétlés - Bináris keresőfa láncolva

Node
+ $key : \mathcal{T}$ // $\mathcal{T}$ valamilyen ismert típus + $left, right$: Node*
+ Node() { $left := right := \oslash$ } // egycsúcsú fát képez belőle + Node($x : \mathcal{T}$) { $left := right := \oslash$; $key := x$ }

Node3
+ $key : \mathcal{T}$ // $\mathcal{T}$ valamilyen ismert típus + $left, right, parent$: Node3*
+ Node3(p:Node3*) { $left := right := \oslash$; $parent := p$ } + Node3($x : \mathcal{T}$, p:Node3*) { $left := right := \oslash$; $parent := p$; $key := x$ }

Ismétlés - Keresés algoritmusa





Miért jó ez nekünk?

Keresés műveletigénye: O(h(t))

Ha a fa sűrű, akkor n csúcs esetén h(t) = O(log(n))

AVL fa

Olyan Bináris keresőfa, amely minden x csúcsára igaz az, hogy:

$$|h(bal(x)) - h(jobb(x))| \leq 1$$

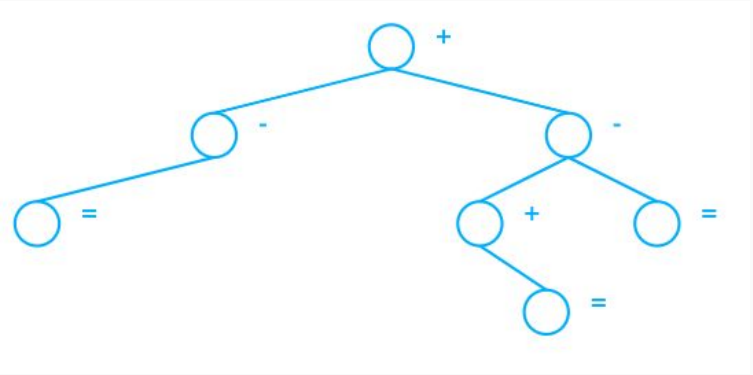
A műveletek változatlanul érvényesek, ugyanúgy kell elvégezni őket. De ha elromlik az AVL tulajdonság azt helyre kell állítani.

AVL tulajdonság tárolása

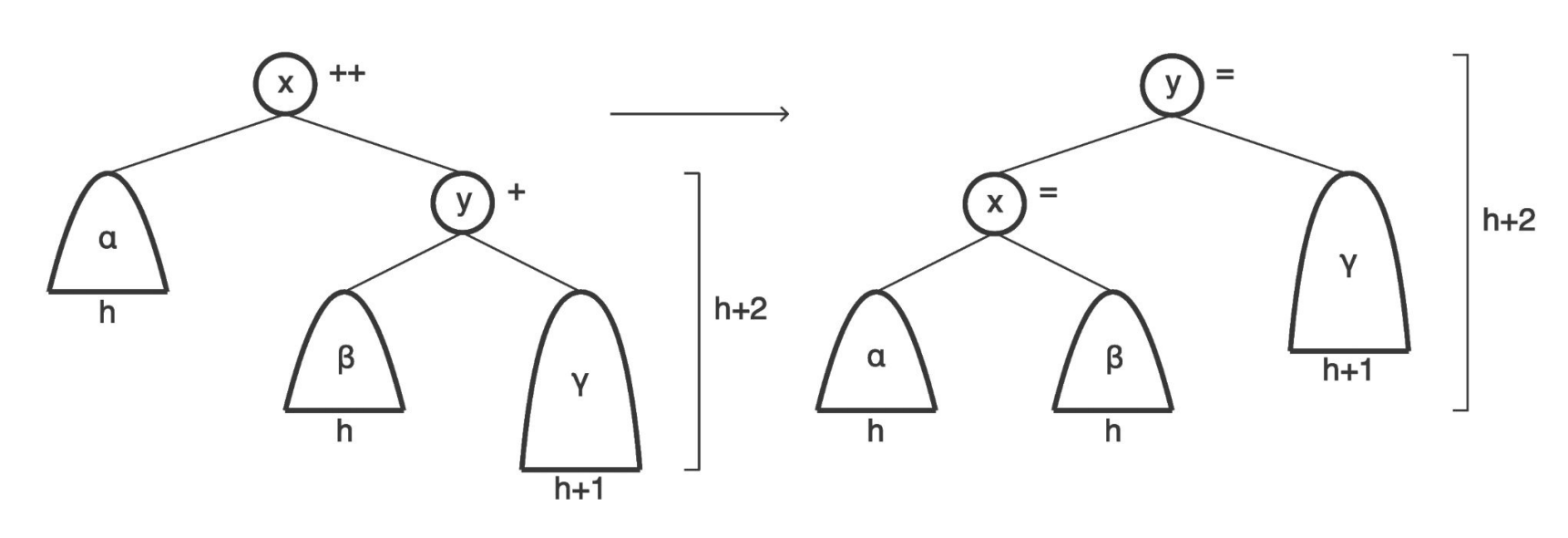
Node3 csúcsokat alkalmazunk és kiegészítjük őket egy plusz adatmezővel, aminek neve legyen hdiff, értéke pedig minden x csúcsra.

x.hdiff = h(jobb(x)) - h(bal(x))

Ezt címkékkel jelezzük a fában.
A megengedett értékek: +,-,0



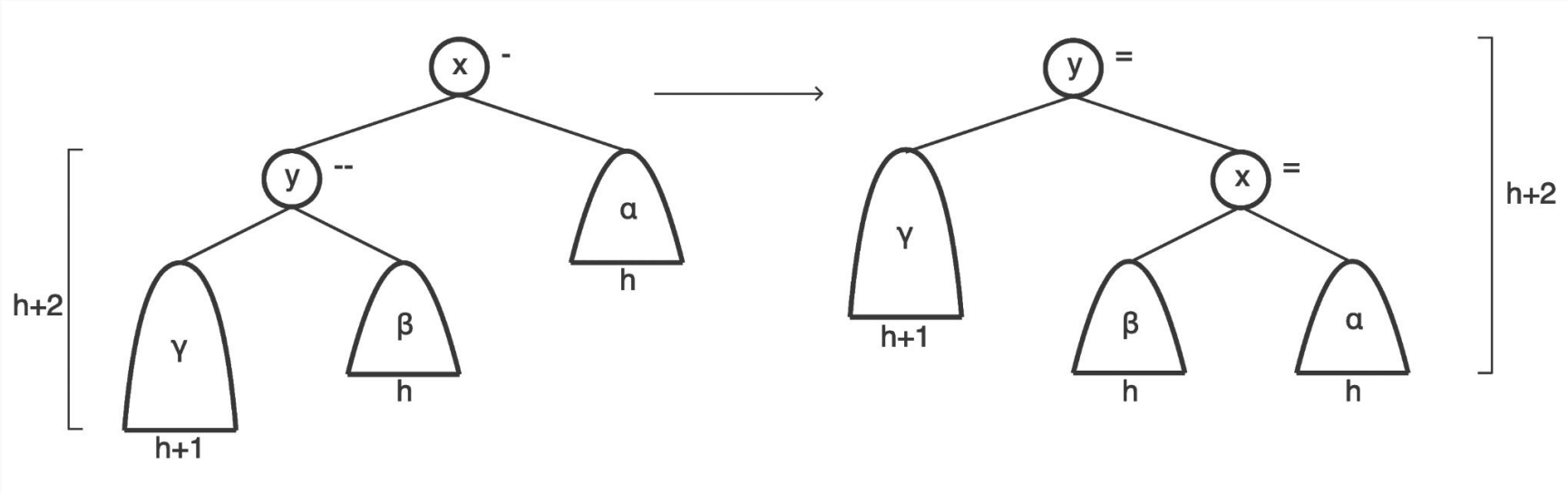
AVL forgatások - (++,+) (--,-)



4. Feladat

Rajzoljuk fel a (--,-) szabályt, amely a (++,+) szabály tükörképe!

Megoldás



Beszúrás algoritmusa

1. Megkeressük a kulcs helyét a fában, ha a kulcs benne van a fában, akkor KÉSZ vagyunk.
2. Ha a kulcs helyén egy üres részfa található, beszúrunk az üres fa helyére egy új a kulcsot tartalmazó levélcsúcsot, azzal, hogy ez a részfa eggyel magasabb lett.
3. Egyet felfelé lépünk a keresőfában. Mivel az a részfa, amiből felfelé építünk, eggyel magasabb lett, az aktuális csúcs egyensúlyát megfelelően módosítjuk. (Ha a jobb részfa lett magasabb, hozzáadunk az egyensúlyhoz egyet, ha a bal, levonunk belőle egyet.)
4. Ha az aktuális csúcs egyensúlya 0 lett, akkor az aktuális csúcshoz tartozó részfa alacsonyabb ága hozzánőtt a magasabbhoz, tehát az aktuális részfa most ugyanolyan magas, mint a beszúrás előtt volt, és így egyetlen más csúcs egyensúlyát sem kell módosítani, KÉSZ vagyunk.
5. Ha az aktuális csúcs új egyensúlya 1 vagy -1, akkor előtte 0 volt, ezért az aktuális részfa magasabb lett eggyel. Ekkor a 4. ponttól folytatjuk.
6. Ha az aktuális csúcs új egyensúlya 2 vagy -2, akkor a hozzá tartozó részfát ki kell egyensúlyozni. A kiegyensúlyozás után az aktuális részfa visszanyeri a beszúrás előtti magasságát, ezért már egyetlen más csúcs egyensúlyát sem kell módosítani, KÉSZ vagyunk.

6. Feladat

Építsünk AVL fát az alábbi adatokból:

6, 4, 3, 9, 12, 2, 8

Megoldás

